App Kit

App Kit SDK reference

Copy page

API reference for App Kit's public interfaces, methods, and types

This reference guide describes the public interfaces, methods, and types available in the App Kit SDK.

AppKit Class

The `AppKit` is how you'll perform all stablecoin operations including crosschain bridging, same-chain swaps, token transfers, and fee estimation. It also enables you to add event listeners for bridge transfers.

constructor(config?)

Creates a new `AppKit` instance.

```
constructor(config?: AppKitConfig)
```

Parameters

Name	Type	Description
config	`AppKitConfig`	Optional context overrides used for fee estimation utilities

Usage

```
import { AppKit } from "@circle-fin/app-kit";

const kit = new AppKit();
```

AppKitConfig

```
type AppKitConfig = CreateContextParams & {
  developerFee?: Partial<DeveloperFeeHooks>;
};
```

Methods

bridge(params)

Execute a crosschain USDC bridge transfer.

Transfers USDC between different blockchain networks using Circle's Cross-Chain Transfer Protocol (CCTP). Supports both fast and standard transfer speeds with automatic attestation handling.

```
async bridge(params: BridgeParams<AdapterCapabilities, AdapterCapabilities>): B
```

Parameters

Name	Type	Description
params	`BridgeParams<AdapterCapabilities, AdapterCapabilities>`	Bridge parameters containing source, destination, amount, and token

Returns

BridgeResult

Usage Example

```
const result = await kit.bridge({
  from: sourceAdapter,
  to: { adapter: destAdapter, chain: "Polygon" },
  amount: "100.50",
  token: "USDC",
});

console.log("Bridge completed:", result.hash);
```

---

estimateBridge(params)

Estimate the bridge operation.

Calculates gas costs, protocol fees, and optional custom fees for a crosschain bridge transfer without executing the transaction. Useful for displaying cost estimates to users before they confirm a transfer.

```
async estimateBridge(params: BridgeParams<AdapterCapabilities, AdapterCapabilit
```

Parameters

Name	Type	Description
params	`BridgeParams<AdapterCapabilities, AdapterCapabilities>`	Bridge parameters containing source, destination, amount, and token

Returns

EstimateResult

Usage Example

```
const estimate = await kit.estimateBridge({
  from: sourceAdapter,
  to: { adapter: destAdapter, chain: "Polygon" },
  amount: "100.50",
  token: "USDC",
});

console.log("Estimated fee:", estimate.fee);
```

---

estimateSend(params)

Estimate network fees for a send operation.

Prepare the send (validation + recipient resolution) and returns the gas estimate without executing the actual transaction. This allows developers to show users the cost before committing to the transfer.

```
async estimateSend(params: SendParams): EstimatedGas
```

Parameters

Name	Type	Description
params	`SendParams`	Send parameters: source, destination (address or adapter), amount, token.

Returns

EstimatedGas

Usage Examples

```
const estimate = await kit.estimateSend({
  from: sourceAdapter,
  to: { adapter: destAdapter, chain: "Polygon" },
  amount: "100.50",
  token: "USDC",
});

console.log("Estimated gas:", estimate.gas);
```

```
const estimate = await kit.estimateSend({
  from: sourceAdapter,
  to: { adapter: destAdapter, chain: "Polygon" },
  amount: "50.0",
  token: "USDT",
});

console.log("Estimated gas:", estimate.gas);
```

---

estimateSwap(params)

Estimate the output and fees for a swap operation.

Calculates the expected output amount, minimum output (with slippage), and fee breakdown for a token swap without executing the transaction.

```
async estimateSwap(params: SwapParams<AdapterCapabilities>): SwapEstimate
```

Parameters

Name	Type	Description
params	`SwapParams<AdapterCapabilities>`	Swap parameters containing source, tokens, amount, and config

Returns

SwapEstimate

›

Usage Example

```
const estimate = await kit.estimateSwap({
  from: { adapter, chain: "Ethereum" },
  tokenIn: "USDC",
  tokenOut: "USDT",
  amountIn: "100.50",
  config: {
    slippageBps: 300,
    kitKey: "KIT_KEY:id:secret",
  },
});

console.log("Stop limit:", estimate.stopLimit.amount, estimate.stopLimit.token)
console.log(
  "Estimated output:",
  estimate.estimatedOutput.amount,
  estimate.estimatedOutput.token,
);
console.log("Fees:", estimate.fees);
```

---

getSupportedChains(operationType)

Get chains supported by AppKit operations.

Returns blockchain networks that support specific stablecoin operations. When no operation type is specified, returns all chains supporting any operation (bridge or swap).

```
getSupportedChains(operationType: OperationType): ChainDefinition[]
```

Parameters

Name	Type	Description
operationType	`OperationType`	Optional operation type to filter chains (`bridge` | `swap`)

Returns

ChainDefinition[]

Usage Examples

```
import { AppKit } from "@circle-fin/app-kit";

const kit = new AppKit();
const allChains = kit.getSupportedChains();

console.log(`Total supported chains: ${allChains.length}`);
allChains.forEach((chain) => {
  console.log(`- ${chain.name} (${chain.type})`);
});
```

```
const kit = new AppKit();
const bridgeChains = kit.getSupportedChains("bridge");

console.log(
  "Chains supporting bridge:",
  bridgeChains.map((c) => c.name),
);
```

---

off(action)

Unregister an event handler for a specific AppKit action.

This method removes a previously registered event handler. You must pass the exact same handler function reference that was used during registration. Use the wildcard `*` to remove handlers listening to all actions.

```
off(action: K, handler: (payload: any) => void): void
```

Parameters

Name	Type	Description
action	`K`	The action name (prefixed with `bridge.`) or `*` for all actions
handler	`(payload: any) => void`	The handler function to remove (must be the same reference)

Usage Example

```
import { AppKit } from "@circle-fin/app-kit";

const kit = new AppKit();

// Define handler
const handler = (payload) => {
  console.log("Approval:", payload);
};

// Register
kit.on("bridge.approve", handler);

// Later, unregister
kit.off("bridge.approve", handler);
```

---

on(action)

Register an event handler for a specific AppKit action.

Subscribe to events emitted during bridge operations. All bridge events are prefixed with `bridge.` (e.g., `bridge.approve` , `bridge.burn`) to namespace them within the AppKit event system.

Handlers receive strongly-typed payloads based on the action name. Multiple handlers can be registered for the same action, and all will be invoked when the action occurs. Use the wildcard `*` to listen to all actions.

```
on(action: K, handler: (payload: any) => void): void
```

Parameters

Name	Type	Description
action	`K`	The action name (prefixed with `bridge.`) or `*` for all actions
handler	`(payload: any) => void`	Callback function to invoke when the action occurs

Usage Example

```
import { AppKit } from "@circle-fin/app-kit";

const kit = new AppKit();

// Listen to specific bridge action
kit.on("bridge.approve", (payload) => {
  console.log("Approval transaction:", payload.values.txHash);
});

// Listen to all actions
kit.on("*", (payload) => {
  console.log("Action:", payload.method);
});
```

---

send(params)

Execute a send operation for known token aliases (USDC, USDT, NATIVE) or custom ERC-20/SPL tokens. For custom tokens, the token address must be provided.

This method handles the complete send transfer flow using the underlying AppKit infrastructure. It supports sending to either a destination adapter or an explicit recipient address, with full type safety and comprehensive error handling.

```
async send(params: SendParams): BridgeStep
```

Parameters

Name	Type	Description
params	`SendParams`	Send parameters containing source, destination, amount, and token

Returns

BridgeStep

Usage Examples

```
// Send USDC to another adapter
const result = await kit.send({
  from: sourceAdapter,
  to: { adapter: destAdapter, chain: "Polygon" },
  amount: "100.50",
  token: "USDC",
});

console.log("Send completed:", result.txHash);
```

```
// Send a custom token to an explicit address
const result = await kit.send({
  from: sourceAdapter,
  to: "0x742d35Cc4634C0532925a3b8D1d7",
  amount: "100.50",
  token: "0x6B175474E89094C44Da98b954EedeAC495271d0F", // DAI on Ethereum
});
console.log("Send completed:", result.txHash);
```

```
// Send USDT to an explicit address
const result = await kit.send({
  from: sourceAdapter,
  to: { address: "0x742d35Cc4634C0532925a3b8D1d7", chain: "Polygon" },
  amount: "50.25",
  token: "USDT",
});

console.log("Send completed:", result.txHash);
```

---

swap(params)

Execute a same-chain token swap operation.

Swaps between USDC, USDT, and native tokens on the same blockchain with configurable slippage tolerance and allowance strategies.

```
async swap(params: SwapParams<AdapterCapabilities>): SwapResult
```

Parameters

Name	Type	Description
params	`SwapParams<AdapterCapabilities>`	Swap parameters containing source, tokens, amount, and config

Returns

SwapResult

Usage Example

```
const result = await kit.swap({
  from: { adapter, chain: "Ethereum" },
  tokenIn: "USDC",
  tokenOut: "USDT",
  amountIn: "100.50",
  config: {
    slippageBps: 300, // 3% slippage
    allowanceStrategy: "permit",
    kitKey: "KIT_KEY:id:secret",
  },
});

console.log("Swap completed:", result.txHash);
```

Method Parameters

BridgeParams

Parameters for initiating a crosschain USDC bridge transfer.

This type is used as the primary input to BridgeKit.bridge, allowing users to specify the source and destination adapters, transfer amount, and optional configuration.

- The `from` field specifies the source adapter context (wallet and chain).
- The `to` field specifies the destination, supporting both explicit and derived recipient addresses.

- The `config` field allows customization of bridge behavior (e.g., transfer speed).
- The `token` field is optional and defaults to `USDC`; other tokens are not currently supported.

```
interface BridgeParams {
  amount: string;
  config?: BridgeConfig;
  from: AdapterContext<TFromAdapterCapabilities, BridgeChainIdentifier>;
  invocationMeta?: InvocationMeta;
  to: BridgeDestination<TToAdapterCapabilities, BridgeChainIdentifier>;
  token?: "USDC";
}
```

Properties

Name	Type	Description
amount	string	The amount to transfer
config	BridgeConfig	Optional bridge configuration (e.g., transfer speed). If omitted, defaults will be used
from	`AdapterContext<TFromAdapterCapabilities, BridgeChainIdentifier>`	The source adapter context (wallet and chain) for the transfer.
invocationMeta	InvocationMeta	Optional invocation metadata for tracing and correlation. When provided, the `traceId` is used to correlate all events emitted during the bridge operation. If not provided, an OpenTelemetry-compatible `traceId` will be auto-generated.
to	`BridgeDestination<TToAdapterCapabilities, BridgeChainIdentifier>`	The destination for the transfer, supporting explicit or derived recipient addresses

Name	Type	Description
token	`'USDC'`	The token to transfer. Defaults to `USDC`. If omitted, the provider will use `USDC` by default.

BridgeConfig

Configuration options for customizing bridge behavior.

```
interface BridgeConfig {
  customFee?: CustomFee
  maxFee?: string
  transferSpeed?: TransferSpeed \| 'FAST' \| 'SLOW'
}
```

Properties

Name	Type	Description
customFee	CustomFee	The custom fee to charge for the transfer. Whatever value you provide here is added on top of the transfer amount. The user must have enough balance for `amount + customFee`, and the wallet signs for that total on the source chain. The custom fee is split automatically: - 10% routes to Circle. - 90% routes to your `recipientAddress`. The original transfer amount proceeds through CCTPv2 unchanged, and the protocol fee (1–14 bps in FAST mode, 0% in STANDARD) is taken from that transfer amount.
maxFee	string	The maximum fee to pay for the burn operation.

Name	Type	Description
›		Provide the amount as a base-10 numeric string representing the token amount in human-readable format. For example: to set a maximum fee of 1 USDC, pass `"1"`. Decimal values are supported (e.g., `"0.5"` for half a USDC).
transferSpeed	`TransferSpeed | 'FAST' | 'SLOW'`	The transfer speed mode for CCTPv2 transfers. Controls whether to use fast burn mode (FAST) or standard mode (SLOW). Fast burn may reduce transfer time but could have different fee implications.

AdapterContext

Represents the context of an adapter used for crosschain operations.

An AdapterContext must always specify both the adapter and the chain explicitly. The address field behavior is determined by the adapter's address control model:

- Developer-controlled adapters: The `address` field is required because each operation must explicitly specify which address to use.
- User-controlled adapters: The `address` field is forbidden because the address is automatically resolved from the connected wallet or signer.
- Legacy adapters: The `address` field remains optional for backward compatibility.

This ensures clear, debuggable code where the intended chain is always visible at the call site, and address requirements are enforced at compile time based on adapter capabilities.

```
type AdapterContext = {
  adapter: Adapter<TAdapterCapabilities>;
  chain: TChainIdentifier;
} & AddressField<ExtractAddressContext<TAdapterCapabilities>>;
```

CCTPConfig

Configuration for the Cross-Chain Transfer Protocol (CCTP).

```
interface CCTPConfig {
  contracts: Partial<object>;
  domain: number;
  forwarderSupported: object;
}
```

Properties

Name	Type	Description
contracts	`Partial<object>`	The contracts required for CCTP.
domain	number	The CCTP domain identifier.
forwarderSupported	object	Indicates whether the chain supports forwarder for source and destination.

VersionConfig

Version configuration for CCTP contracts.

Defines whether the chain uses split or merged CCTP contract architecture. Split configuration uses separate TokenMessenger and MessageTransmitter contracts, while merged configuration uses a single unified contract.

```
type VersionConfig = CCTPSplitConfig | CCTPMergedConfig;
```

SendParams

Parameters for sending USDC, USDT, native tokens, or custom ERC-20/SPL tokens.

This interface is the canonical input for send operations in App Kit. It supports sending to either a destination AdapterContext (recipient derives from the adapter's default account) or an explicit recipient `string` address on the destination chain.

- The `from` field provides the source signing context and chain.
- The `to` field identifies the destination as an adapter context or an explicit address.
- The `amount` field is a human-readable decimal string (for example, `10.5`).
- The `token` field selects the asset to move and defaults to `USDC`.

```
interface SendParams {
  amount: string
  from: AdapterContext
  to: string \| Adapter<AdapterCapabilities>
  token?: TokenAlias \| TokenAddress
}
```

Properties

Name	Type	Description
amount	string	The amount to transfer.
from	AdapterContext	The source adapter context (wallet and chain) for the transfer.
to	`string | Adapter<AdapterCapabilities>`	The destination for the transfer, supporting explicit or derived recipient addresses.
token	`TokenAlias | TokenAddress`	The token to transfer. Defaults to `USDC`. If omitted, the provider will use `USDC` by default. Supports both known aliases and custom token contract addresses: - Known aliases: `USDC`, `USDT`, `NATIVE` - Custom token addresses: EVM addresses or Solana SPL token mint addresses

SwapParams

Parameters for initiating a same-chain stablecoin swap.

This type is used as the primary input to swap operations, allowing users to specify the source context, input/output tokens, swap amount, and optional configuration.

SwapKit supports swaps between stablecoins (USDC, USDT, EURC, USDe, DAI, PYUSD), wrapped tokens (WBTC, WETH, WSOL, WAVAX, WPOL), and native tokens (e.g., ETH on Ethereum, SOL on Solana) via the `NATIVE` alias.

- The `from` field specifies the source adapter context (wallet and chain). The chain must be a SwapChain member (or its corresponding string literal) so IDE autocomplete only surfaces swap-supported networks.
- The `tokenIn` field specifies the input token (see SupportedToken).
- The `tokenOut` field specifies the output token (see SupportedToken).
- The `amountIn` field is a human-readable decimal string (e.g., `10.5`).
- The `config` field allows customization of swap behavior.

```
interface SwapParams {
  amountIn: string
  config?: SwapConfig
  from: SwapAdapterContext<TFromAdapterCapabilities>
  tokenIn: infer<any>
  tokenOut: infer<any>
}
```

Properties

Name	Type	Description
amountIn	string	The amount of the input token to swap. Expressed as a human-readable decimal string in token units (e.g., `0.05` for 0.05 USDC or 0.05 ETH).

Name	Type	Description
config	SwapConfig	Optional configuration for swap behavior. If omitted, defaults will be used: - `allowanceStrategy` : `permit` (fallback to `approve`) - `slippageBps` : 300 (3%)
from	`SwapAdapterContext<TFromAdapterCapabilities>`	The source adapter context (wallet and chain) for the swap.
tokenIn	`infer<any>`	The input token to swap from. Supports stablecoins (USDC, USDT, EURC, USDe, DAI, PYUSD), wrapped tokens (WBTC, WETH, WSOL, WAVAX, WPOL), and native tokens (NATIVE or chain-specific symbols).
tokenOut	`infer<any>`	The output token to swap to. Supports stablecoins (USDC, USDT, EURC, USDe, DAI, PYUSD), wrapped tokens (WBTC, WETH, WSOL, WAVAX, WPOL), and native tokens (NATIVE or chain-specific symbols).

SwapConfig

Configuration options for swap operations.

Controls swap behavior including allowance strategy, slippage tolerance, minimum output amounts, custom fees, and kit identification.

```
interface SwapConfig {
  allowanceStrategy?: AllowanceStrategy;
  customFee?: object;
  kitKey?: string;
  slippageBps?: number;
  stopLimit?: string;
}
```

Properties

Name	Type	Description
allowanceStrategy	AllowanceStrategy	Strategy for granting token allowances to the swap contract. Defaults to `permit` with fallback to `approve`.
customFee	object	Custom fee configuration for this swap (percentage-based approach). Allows specifying a percentage fee and recipient address at the transaction level. This is mutually exclusive with kit-level callback fee policy. If both are set, transaction-level takes precedence. For complex fee logic (VIP tiers, database lookups), use kit-level callback approach via `setCustomFeePolicy()` instead.
kitKey	string	Identifier for the kit instance initiating this swap. Used for tracking and analytics purposes.
slippageBps	number	Maximum acceptable slippage in basis points (BPS). 1 BPS = 0.01%, so 300 BPS = 3% slippage. Defaults to 300 BPS (3%).
stopLimit	string	Minimum acceptable output amount in human-readable format (stop-limit).

Name	Type	Description
›		If the estimated output falls below this value, the swap will fail. Expressed as a decimal string (e.g., `0.4` for 0.4 USDT). The value is automatically converted to base units using the tokenOut decimals.

CCTPSplitConfig

Split CCTP contract configuration.

Used by chains that deploy separate TokenMessenger and MessageTransmitter contracts. This is the traditional CCTP architecture used by most EVM chains.

```
interface CCTPSplitConfig {
  confirmations: number;
  messageTransmitter: string;
  tokenMessenger: string;
  type: "split";
}
```

CCTPMergedConfig

Merged CCTP contract configuration.

Used by chains that deploy a single unified CCTP contract. This simplified architecture is used by newer chain integrations.

```
interface CCTPMergedConfig {
  confirmations: number;
  contract: string;
  type: "merged";
}
```

Method Results

BridgeResult

Result object returned after a successful crosschain bridge operation.

This interface contains all the details about a completed bridge, including the bridge parameters, source and destination information, and the sequence of steps that were executed.

```
interface BridgeResult {
  amount: string
  config?: BridgeConfig
  destination: object
  provider: string
  source: object
  state: 'pending' \| 'success' \| 'error'
  steps: BridgeStep[]
  token: 'USDC'
}
```

Properties

Name	Type	Description
amount	string	The amount that was transferred (as a string to avoid precision issues)
config	BridgeConfig	The bridge configuration that was used for this operation
destination	object	Information about the destination chain and address
provider	string	The provider that was used for this operation
source	object	Information about the source chain and address
state	`'pending' | 'success' | 'error'`	The state of the transfer

Name	Type	Description
steps	BridgeStep[]	Array of steps that were executed during the bridge process
token	`'USDC'`	The token that was transferred (currently only USDC is supported)

BridgeStep

A step in the bridge process.

```
interface BridgeStep {
  data?: unknown
  error?: unknown
  errorMessage?: string
  explorerUrl?: string
  forwarded?: boolean
  name: string
  state: 'pending' \| 'success' \| 'error' \| 'noop'
  txHash?: string
}
```

Properties

Name	Type	Description
data	unknown	Optional data for the step
error	unknown	Optional raw error object (can be Viem/Ethers/Chain error)
errorMessage	string	Optional human-readable error message
explorerUrl	string	Optional explorer URL for viewing this transaction on a block explorer
forwarded	boolean	Whether this step was executed via Circle's Forwarder (relay service). Only applicable

Name	Type	Description
		for mint steps.
›		- `true` : The mint was handled by Circle's Orbit relayer - `false` : The user submitted the mint transaction directly - `undefined` : Not applicable (non-mint steps)
name	string	Human-readable name of the step (e.g., "Approve", "Burn", "Mint")
state	`'pending' | 'success' | 'error' | 'noop'`	The state of the step
txHash	string	Optional transaction hash for this step (if applicable)

EstimateResult

Cost estimation result for a crosschain transfer operation.

This interface provides detailed information about the expected costs for a transfer, including gas fees on different chains and protocol fees. It also includes the input context (token, amount, source, destination) to provide a complete view of the transfer being estimated.

```
interface EstimateResult {
  amount: string;
  destination: object;
  fees: object[];
  gasFees: object[];
  source: object;
  token: "USDC";
}
```

Properties

Name	Type	Description
amount	string	The amount being transferred
destination	object	Information about the destination chain and address
fees	object[]	Array of protocol and service fees for the transfer
gasFees	object[]	Array of gas fees required for the transfer on different blockchains
source	object	Information about the source chain and address
token	`'USDC'`	The token being transferred

EstimatedGas

Estimated gas information for a blockchain transaction.

This interface provides a unified way to represent gas costs across different blockchain networks, supporting both EVM-style gas calculations and other fee models.

```
interface EstimatedGas {
  fee: string;
  gas: bigint;
  gasPrice: bigint;
}
```

SwapEstimate

Estimation result for a swap operation.

Contains the provider's swap quote including minimum output (stop limit), estimated output amount, fee breakdown, and input context fields

```
interface SwapEstimate {
  amountIn: string
  chain: Blockchain
  estimatedOutput: TokenAmount
  fees?: any
  fromAddress: string
  stopLimit: TokenAmount
  toAddress: string
  tokenIn: infer<any>
  tokenOut: infer<any>
}
```

Properties

Name	Type	Description
amountIn	string	The input amount that will be swapped. Expressed as a human-readable decimal string in token units (e.g., `0.05` for 0.05 USDC or 0.05 ETH).
chain	Blockchain	The chain where the swap will be executed. Returns the chain name (e.g., Blockchain.Ethereum) Use `getChainByEnum(chain)` to resolve back to a full ChainDefinition if needed.
estimatedOutput	TokenAmount	Estimated output amount with token information.
fees	any	Detailed fee breakdown for the swap operation.
fromAddress	string	The address that will initiate the swap.
stopLimit	TokenAmount	Estimated minimum token out amount with token information. This represents the minimum amount of tokens the user should receive after accounting for slippage. Amount is in human-readable decimal format.
toAddress	string	The address that will receive the swapped tokens.

Name	Type	Description
tokenIn	`infer<any>`	The input token that will be swapped from.
tokenOut	`infer<any>`	The output token that will be swapped to.

SwapResult

Result of an executed swap operation.

Captures the outcome of a swap transaction, including input/output tokens, transaction hash, and fee information. This is the high-level result returned to users after a swap operation completes.

```
interface SwapResult {
  amountIn: string
  amountOut?: string
  chain: Blockchain
  config?: SwapResultConfig
  explorerUrl?: string
  fees?: any
  fromAddress: string
  toAddress: string
  tokenIn: infer<any>
  tokenOut: infer<any>
  txHash: string
}
```

Properties

Name	Type	Description
amountIn	string	The input amount that was swapped. Expressed as a human-readable decimal string in token units (e.g., `0.05` for 0.05 USDC or 0.05 ETH).

Name	Type	Description
›		The actual output amount received from the completed swap.
amountOut	string	Expressed as a human-readable decimal string in output token units. Populated via a best-effort LI.FI status lookup after the on-chain transaction confirms. May be `undefined` if the lookup fails or the chain type is unsupported.
chain	Blockchain	The chain where the swap was executed. Returns the chain name (e.g., Blockchain.Ethereum) Use `getChainByEnum(chain)` to resolve back to a full ChainDefinition if needed.
config	SwapResultConfig	The swap configuration that was used for this operation.
explorerUrl	string	The formatted explorer URL for viewing this transaction on a block explorer. Only present when `txHash` is non-empty.
fees	any	Detailed fee breakdown for the swap operation. Includes both provider fees (charged by the DEX aggregator/protocol) and kit fees (if configured).
fromAddress	string	The address that initiated the swap.
toAddress	string	The address that received the swapped tokens.
tokenIn	`infer<any>`	The input token that was swapped from.
tokenOut	`infer<any>`	The output token that was swapped to.
txHash	string	The transaction hash for the executed swap. Available once the transaction has been submitted to the network.

Supporting Types

TransferSpeed

Transfer speed options for crosschain operations.

Defines the available speed modes for CCTPv2 transfers, affecting both transfer time and potential fee implications.

```
enum TransferSpeed {
  FAST = 'FAST'
  SLOW = 'SLOW'
}
```

Values

`FAST` , `SLOW`

ChainDefinition

Public chain definition type.

```
type ChainDefinition = EVMChainDefinition | NonEVMChainDefinition;
```

EVMChainDefinition

Represents chain definitions for Ethereum Virtual Machine (EVM) compatible blockchains.

```
interface EVMChainDefinition {
  cctp: null \| CCTPConfig
  chain: Blockchain
  chainId: number
  eurcAddress: null \| string
  explorerUrl: string
  isTestnet: boolean
  kitContracts?: Partial<Record<KitContractType, string>>
  name: string
  nativeCurrency: Currency
  rpcEndpoints: any
  title?: string
  type: 'evm'
  usdcAddress: null \| string
  usdtAddress: null \| string
}
```

Properties

Name	Type	Description
cctp	`null | CCTPConfig`	Optional CCTP configuration.
chain	Blockchain	The blockchain identifier from the Blockchain enum.
chainId	number	The unique identifier for the blockchain.
eurcAddress	`null | string`	The contract address for EURC.
explorerUrl	string	Template URL for the blockchain explorer to view transactions.
isTestnet	boolean	Indicates whether this is a testnet or mainnet.
kitContracts	`Partial<Record<KitContractType, string>>`	Optional kit-specific contract addresses for enhanced chain functionality.
name	string	The display name of the blockchain.

Name	Type	Description
nativeCurrency ›	Currency	Information about the native currency of the blockchain.
rpcEndpoints	any	Default RPC endpoints for connecting to the blockchain network.
title	string	Optional title or alternative name for the blockchain.
type	`'evm'`	Discriminator for EVM chains.
usdcAddress	`null | string`	The contract address for USDC.
usdtAddress	`null | string`	The contract address for USDT.

Blockchain

Enumeration of all blockchains known to this library.

This enum contains every blockchain that has a chain definition, regardless of whether bridging is currently supported. For chains that support bridging via CCTPv2, see BridgeChain.

```
enum Blockchain {
  Algorand = 'Algorand'
  Algorand_Testnet = 'Algorand_Testnet'
  Aptos = 'Aptos'
  Aptos_Testnet = 'Aptos_Testnet'
  Arbitrum = 'Arbitrum'
  Arbitrum_Sepolia = 'Arbitrum_Sepolia'
  Arc_Testnet = 'Arc_Testnet'
  Avalanche = 'Avalanche'
  Avalanche_Fuji = 'Avalanche_Fuji'
  Base = 'Base'
  Base_Sepolia = 'Base_Sepolia'
  Celo = 'Celo'
  Celo_Alfajores_Testnet = 'Celo_Alfajores_Testnet'
  Codex = 'Codex'
  Codex_Testnet = 'Codex_Testnet'
  Ethereum = 'Ethereum'
  Ethereum_Sepolia = 'Ethereum_Sepolia'
  Hedera = 'Hedera'
  Hedera_Testnet = 'Hedera_Testnet'
  HyperEVM = 'HyperEVM'
  HyperEVM_Testnet = 'HyperEVM_Testnet'
  Ink = 'Ink'
  Ink_Testnet = 'Ink_Testnet'
  Linea = 'Linea'
  Linea_Sepolia = 'Linea_Sepolia'
  Monad = 'Monad'
  Monad_Testnet = 'Monad_Testnet'
  NEAR = 'NEAR'
  NEAR_Testnet = 'NEAR_Testnet'
  Noble = 'Noble'
  Noble_Testnet = 'Noble_Testnet'
  Optimism = 'Optimism'
  Optimism_Sepolia = 'Optimism_Sepolia'
  Plume = 'Plume'
  Plume_Testnet = 'Plume_Testnet'
  Polkadot_Asset_Hub = 'Polkadot_Asset_Hub'
  Polkadot_Westmint = 'Polkadot_Westmint'
  Polygon = 'Polygon'
  Polygon_Amoy_Testnet = 'Polygon_Amoy_Testnet'
  Sei = 'Sei'
  Sei_Testnet = 'Sei_Testnet'
  Solana = 'Solana'
```

```
  Solana_Devnet = 'Solana_Devnet'
  Sonic = 'Sonic'
  Sonic_Testnet = 'Sonic_Testnet'
  Stellar = 'Stellar'
  Stellar_Testnet = 'Stellar_Testnet'
  Sui = 'Sui'
  Sui_Testnet = 'Sui_Testnet'
  Unichain = 'Unichain'
  Unichain_Sepolia = 'Unichain_Sepolia'
  World_Chain = 'World_Chain'
  World_Chain_Sepolia = 'World_Chain_Sepolia'
  XDC = 'XDC'
  XDC_Apothem = 'XDC_Apothem'
  ZKSync_Era = 'ZKSync_Era'
  ZKSync_Sepolia = 'ZKSync_Sepolia'
}
```

Values

Algorand , Algorand_Testnet , Aptos , Aptos_Testnet , Arbitrum , Arbitrum_Sepolia , Arc_Testnet , Avalanche , Avalanche_Fuji , Base , Base_Sepolia , Celo , Celo_Alfajores_Testnet , Codex , Codex_Testnet , Ethereum , Ethereum_Sepolia , Hedera , Hedera_Testnet , HyperEVM , HyperEVM_Testnet , Ink , Ink_Testnet , Linea , Linea_Sepolia , Monad , Monad_Testnet , NEAR , NEAR_Testnet , Noble , Noble_Testnet , Optimism , Optimism_Sepolia , Plume , Plume_Testnet , Polkadot_Asset_Hub , Polkadot_Westmint , Polygon , Polygon_Amoy_Testnet , Sei , Sei_Testnet , Solana , Solana_Devnet , Sonic , Sonic_Testnet , Stellar , Stellar_Testnet , Sui , Sui_Testnet , Unichain , Unichain_Sepolia , World_Chain , World_Chain_Sepolia , XDC , XDC_Apothem , ZKSync_Era , ZKSync_Sepolia

---

KitContractType

Available kit contract types for enhanced chain functionality.

```
type KitContractType = "bridge" | "adapter";
```

Currency

Represents basic information about a currency or token.

```
interface Currency {
  decimals: number;
  name: string;
  symbol: string;
}
```

NonEVMChainDefinition

Represents chain definitions for non-EVM blockchains.

```
interface NonEVMChainDefinition {
  cctp: null \| CCTPConfig
  chain: Blockchain
  eurcAddress: null \| string
  explorerUrl: string
  isTestnet: boolean
  kitContracts?: Partial<Record<KitContractType, string>>
  name: string
  nativeCurrency: Currency
  rpcEndpoints: any
  title?: string
  type: 'algorand' \| 'avalanche' \| 'solana' \| 'aptos' \| 'near' \| 'stellar'
  usdcAddress: null \| string
  usdtAddress: null \| string
}
```

Properties

Name	Type	Description
cctp	`null | CCTPConfig`	Optional CCTP configuration.

Name	Type	Description
chain	Blockchain	The blockchain identifier from the Blockchain enum.
eurcAddress	`null | string`	The contract address for EURC.
explorerUrl	string	Template URL for the blockchain explorer to view transactions.
isTestnet	boolean	Indicates whether this is a testnet or mainnet.
kitContracts	`Partial<Record<KitContractType, string>>`	Optional kit-specific contract addresses for enhanced chain functionality.
name	string	The display name of the blockchain.
nativeCurrency	Currency	Information about the native currency of the blockchain.
rpcEndpoints	any	Default RPC endpoints for connecting to the blockchain network.
title	string	Optional title or alternative name for the blockchain.
type	`'algorand' | 'avalanche' | 'solana' | 'aptos' | 'near' | 'stellar' | 'sui' | 'hedera' | 'noble' | 'polkadot'`	Discriminator for non-EVM chains.
usdcAddress	`null | string`	The contract address for USDC.
usdtAddress	`null | string`	The contract address for USDT.

AllowanceStrategy

Allowance strategy for token approvals during swap operations.

Defines how token allowances should be granted to the swap contract:

- `permit` : Use EIP-2612 permit signature (gas-efficient, no approval transaction)
- `approve` : Traditional approval transaction

The default strategy is `permit` with fallback to `approve` if permit is not supported.

```
type AllowanceStrategy = "permit" | "approve";
```

OperationType

Union type of all supported operation types in the AppKit.

This type ensures type safety when specifying operation types and enables proper parameter validation based on the selected operation.

```
type OperationType = "bridge" | "swap";
```

DeveloperFeeHooks

```
interface DeveloperFeeHooks {
  getFee: (params: BridgeParams<AdapterCapabilities, AdapterCapabilities>) => b
  getFeeRecipient: (chain: ChainDefinition) => string \| string
}
```

Properties

Name	Type	Description
getFee	`(params: BridgeParams<AdapterCapabilities, AdapterCapabilities>) => bigint | bigint`	Returns the developer fee in USDC's smallest units (e.g. 6 decimals).
getFeeRecipient	`(chain: ChainDefinition) => string | string`	Returns the fee recipient for the given chain.

BaseChainDefinition

Base information that all chain definitions must include.

```
interface BaseChainDefinition {
  cctp: null \| CCTPConfig
  chain: Blockchain
  eurcAddress: null \| string
  explorerUrl: string
  isTestnet: boolean
  kitContracts?: Partial<Record<KitContractType, string>>
  name: string
  nativeCurrency: Currency
  rpcEndpoints: any
  title?: string
  usdcAddress: null \| string
  usdtAddress: null \| string
}
```

Properties

Name	Type	Description
cctp	`null | CCTPConfig`	Optional CCTP configuration.
chain	Blockchain	The blockchain identifier from the Blockchain enum.

Name	Type	Description
eurcAddress	`null | string`	The contract address for EURC.
explorerUrl	string	Template URL for the blockchain explorer to view transactions.
isTestnet	boolean	Indicates whether this is a testnet or mainnet.
kitContracts	`Partial<Record<KitContractType, string>>`	Optional kit-specific contract addresses for enhanced chain functionality.
name	string	The display name of the blockchain.
nativeCurrency	Currency	Information about the native currency of the blockchain.
rpcEndpoints	any	Default RPC endpoints for connecting to the blockchain network.
title	string	Optional title or alternative name for the blockchain.
usdcAddress	`null | string`	The contract address for USDC.
usdtAddress	`null | string`	The contract address for USDT.

TokenInfo

Represents the metadata associated with a token.

```
interface TokenInfo {
  decimals: number;
  name: string;
  symbol: string;
}
```

Event Types

Bridge Events

Bridge events are emitted for each provider in the kit. Events for the built-in CCTP provider follow its transaction steps. You can subscribe to each event multiple times with different callbacks.

Bridge events are prefixed with `bridge.` to namespace them within AppKit:

Event	Description
`bridge.approve`	Token approval transaction completed
`bridge.burn`	Source chain burn transaction completed
`bridge.attestation`	CCTP attestation received
`bridge.mint`	Destination chain mint transaction completed
`*`	Wildcard listener for all events

Usage Example

```
import { AppKit } from "@circle-fin/app-kit";

const kit = new AppKit();

// Listen to specific bridge action
kit.on("bridge.approve", (payload) => {
  console.log("Approval transaction:", payload.values.txHash);
});

// Listen to all actions
kit.on("*", (payload) => {
  console.log("Action:", payload.method);
});
```

Was this page helpful? Yes No

Swap tokens across chains

Account abstraction

Links

Network status

Blog

Community

Arc homepage

Resources

Terms of service

Privacy policy

Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.